# Microtest 64 bit DLL

The Deben "DebenMT64.dll" is a 64 bit DLL that allows access to the Microtest control software (internally this is done via DDE). A C++ header file and simple example console program are included with this documentation.

The DLL is loaded using the standard C++ `LoadLibrary` function, and the individual functions are accessed via `GetProcAddress`. An example code snippet is shown below.

```
#include "stdafx.h"
#include "MTDDE64.h" // Include the Microtest header file
#include <windows.h>
#include <iostream>
#include <conio.h>
using namespace std;

void main() {
// Load the DebenMT64.dll DLL
HINSTANCE hLib=LoadLibrary(L"DebenMT64.dll");
if (hLib==NULL) {
       cout << "Unable to load library!" << endl; _getch(); return; }

// Get the functions from the DLL
MT_Connect    = (DLL_MT_NoParams )GetProcAddress((HMODULE)hLib, "MT_Connect");
MT_Disconnect = (DLL_MT_NoParams )GetProcAddress((HMODULE)hLib, "MT_Disconnect");
MT_StartMotor = (DLL_MT_NoParams )GetProcAddress((HMODULE)hLib, "MT_StartMotor");
MT_StopMotor  = (DLL_MT_NoParams )GetProcAddress((HMODULE)hLib, "MT_StopMotor");
MT_GetForce   = (DLL_MT_Double   )GetProcAddress((HMODULE)hLib, "MT_GetForce");
MT_GotoLoad   = (DLL_MT_SetDouble)GetProcAddress((HMODULE)hLib, "MT_GotoLoad");
// etc...

// Check the functions are not null, i.e we have something to call
if (MT_Connect==NULL) { cout << "Unable to load function(s)." << endl;
       FreeLibrary((HMODULE)hLib); return; }

// Connect to the Microtest (via the 64bit DLL and thus DDE)
MT_Connect();

// Start the motor
MT_StartMotor();

// Display the force
double d = MT_GetForce();
cout << "Force " << d << "N;\t";

// Go to 25N load
MT_GotoLoad(25);
Sleep(5000);

// Stop the motor
MT_StopMotor();

// We've finished, so disconnect from the Microtest
MT_Disconnect();

// Free the DLL
FreeLibrary((HMODULE)hLib);
}
```

The types `DLL_MT_NoParams`, `DLL_MT_Double`, `DLL_MT_Integer`, `DLL_MT_Bool`, `DLL_MT_SetDouble`, `DLL_MT_SetInteger`, `DLL_MT_Version`, and `DLL_MT_Err_Str` are defined in the header file and define the parameters passed / returned from the functions.

The magic number -9999 (`errordouble`) is used to indicate an undefined return value for both integers and floating point numbers.

Notes:

1. "DebenMT64.dll" must be in the same directory as your program
2. This DLL is only for 64 bit Windows
3. If "VCP100D.DLL" is flagged as missing, install VCREDIST_x64.exe from http://www.microsoft.com/en-us/download/details.aspx?id=14632

## DebenMT64.dll Commands

The "DebenMT64.dll" command functions are broken down in to 6 categories – MAIN, INFORMATION, MOTOR, MEASUREMENT, GOTO and TEMPERATURE. These are listed below.

### MAIN commands

#### *void MT_Connect()*

MT_Connect connects to the Microtest (via DDE). This should be the first command called.

#### *void MT_Disconnect()*

MT_Disconnect disconnects from the Microtest. This should be the last command called.

### INFORMATION commands

#### *bool MT_IsConnected()*

MT_IsConnected returns true if connected to the Microtest (via the DebenMicrotest DLL & thus DDE).

#### *void MT_Version(int* Major, Minor)*

MT_Version returns the Major and Minor version numbers for the DLL, e.g. "1.0"

#### *void MT_GetErrorStr (char* Str, int StrLen)*

MT_GetErrorStr returns the last error message that would have been displayed by the Microtest program. The error string will be preceded by a time stamp.

## MOTOR commands

### *bool MT_IsMotorRunning()*

MT_IsMotorRunning returns true (non zero) if the motor is running, false (0) if not.

### *void MT_StartMotor()*

MT_StartMotor asks the Microtest to start the motor using the current motor speed settings, sample rate, etc.; logging of measurements will also begin.

### *void MT_StopMotor()*

MT_StopMotor asks the Microtest to stop the motor but continues recording experimental data. This is useful for measuring relaxation of samples.
Note: Use MT_Stop to stop both the motor and measurements.

### *void MT_MoveApart()*

MT_MoveApart asks the Microtest to move the stage jaws apart (at full speed).

### *void MT_MoveTogether()*

MT_MoveTogether asks the Microtest to move the stage jaws together (at full speed).

### *void MT_SetMotorSpeed(int SpeedNo)*

MT_SetMotorSpeed(Speed 0..4) asks the Microtest to select Motor Speed 0 to 4. The actual speed will depend on your stage and gearbox.

### *double MT_GetMotorSpeed()*

MT_GetMotorSpeed asks the Microtest for the Motor Speed in mm/min otherwise -9999 to indicate an error.

### *double MT_GetMotorSpeedIndex()*

MT_GetMotorSpeed asks the Microtest for the Motor Speed index 0..5 or -9999 to indicate an error.

## MEASUREMENT commands

### *double MT_GetForce()*

MT_GetForce returns the current Force, in Newtons; otherwise -9999 to indicate an error.

### *double MT_GetPosition()*

MT_GetPosition returns the Start Position for the specimen, in mm; otherwise -9999 to indicate an error.

### *double MT_GetExtension()*

MT_GetExtension returns the current Extension, in mm; otherwise -9999 to indicate an error.

### *double MT_GetTime()*

MT_GetTime returns the duration of the current test, in seconds; otherwise -9999 to indicate an error;

### *void MT_Clear()*

MT_Clear asks the Microtest to stop capturing data, clear all existing data and prepare for a new experiment.

### *void MT_SetSampleTime(int SampleTime)*

MT_SetSampleTime sets the sample time for a Goto command (MT_GotoValue).
Sampletime: 0=100ms, 1=200ms, 2=500ms, 3=1s, 4=2s, 5=5s

### *double MT_GetSampleTime()*

MT_GetSampleTime gets the sample time, in seconds.

### *void MT_SetGain(int Gain)*

MT_SetGain(Gain) sets the Gain. Gain: 0=x1, 1=x2, 2=x5, 3=x10, 4=x20, 5=x50.

### *int MT_GetGain()*

MT_GetGain gets the current Gain.

### *MT_AutoZeroLoadCell*

MT_AutoZeroLoadCell asks Microtest to auto-zero the load cell. This usually finishes within a minute, but can take longer – use MT_IsZeroLoadCellRunning to check if it has completed.

### *MT_IsZeroLoadCellRunning*

MT_IsZeroLoadCellRunning asks Microtest if auto-zeroing of the load cell has finished; returns non-zero if running, 0 if finished.

## GOTO commmands

### *void MT_GotoLoad(double Load)*

MT_GotoLoad(Load) requests the Microtest to go to specified Load in Newtons (passed as a double). This is equivalent to pressing the “Goto” button on the Microtest user interface.

### *void MT_GotoAbsolutePosition (double AbsPos)*

MT_GotoAbsolutePosition(AbsPos) requests the Microtest to go to specified Absolute Position in mm (passed as a double).

### *void MT_GotoExtension (double Extension)*

MT_GotoExtension(Ext) requests the Microtest to go to specified Extension in mm (passed as a double).

### *void MT_GotoSeconds(double Seconds)*

MT_GotoSeconds(Secs) requests the Microtest to go to specified Extension in mm (passed as a double).

### *void MT_ActionConstantLoad()*

MT_ConstantLoad asks the Microtest to switch to constant load mode when the goto command has finished.

### *void MT_ActionMeasure()*

MT_Measure asks the Microtest to start measuring when the goto command has finished.

### *void MT_ActionStop()*

MT_Stop asks the Microtest to stop the motors and stop the recording of experimental data when the goto command has finished.

### *void MT_GotoValue(double Value)*

MT_GotoValue(Value) tells Microtest to go to a specific value (specified by the MT_SetGotoUnits command), e.g. 23.45N.

### *void MT_SetGotoAction(int Action)*

MT_SetGotoAction(Action 0..2) tells Microtest what Goto action to perform once it has reached the specified value (MT_GotoValue). Value: 0=Stop, 1=Measure, 2=Constant load.

### *int MT_GetGotoAction()*

MT_GetGotoAction asks Microtest what the Goto action is - 0=Stop, 1=Measure, 2=Constant load, [3=Step & 4=Cyclic].

### *void MT_SetGotoUnits(int GotoUnits)*

MT_SetGotoUnits(Units 0..3) tells Microtest what units the MT_GotoValue command should use. Gotounits: 0=Force (N); 1=Absolute Pos (mm); 2=Extension (mm); 3=Seconds.

### *int MT_GetGotoUnits()*

MT_GetGotoUnits asks Microtest what units the MT_GotoValue command is using - 0=N (Force); 1=mm (Extension); 2=mm (Absolute Pos); 3=Seconds.

## TEMPERATURE commands

### void MT_SetTemperatureA(double TempA)

MT_SetTemperatureA(Temp_C) asks Microtest to set temperature A (top jaws) to the value, in °C. Note: Setting the temperature to -999 will switch off temperature control on this jaw.

### void MT_SetTemperatureB(double TempB)

MT_SetTemperatureA(Temp_C) asks Microtest to set temperature B (bottom jaws) to the value, in °C. Note: Setting the temperature to -999 will switch off temperature control on this jaw.

### *void MT_TemperatureControlOff()*

MT_TemperatureControLOff asks Microtest to switch off temperature control.

### *double MT_GetTemperatureA()*

MT_GetTemperatureA asks Microtest for the current temperature of the upper (A) jaw, in °C.

### *double MT_GetTemperatureB()*

MT_GetTemperatureB asks Microtest for the current temperature of the lower (B) jaw, in °C.

**Microtest 64 bit DLL**

### *int MT_GetStatus()*

MT_GetStatus asks Microtest what the current status is - 0=OK; 999=Over temperature / water failure; -9999=general error.

### *int MT_GetStatusJawTop()*

MT_GetStatusJawTop asks Microtest for the current status of the top jaw - 0=OFF; 1=Heating; 2=Cooling; -9999=error.

### *int MT_GetStatusJawBottom()*

MT_GetStatusJawBottom asks Microtest for the current status of the bottom jaw - 0=OFF; 1=Heating; 2=Cooling; -9999=error.

### *double MT_GetTemperatureHSA()*

MT_GetTemperatureHSA asks Microtest for the current heat sink temperature of the upper (A) jaw, in °C.

### *double MT_GetTemperatureHSB()*

MT_GetTemperatureHSB asks Microtest for the current heat sink temperature of the lower (B) jaw, in °C.